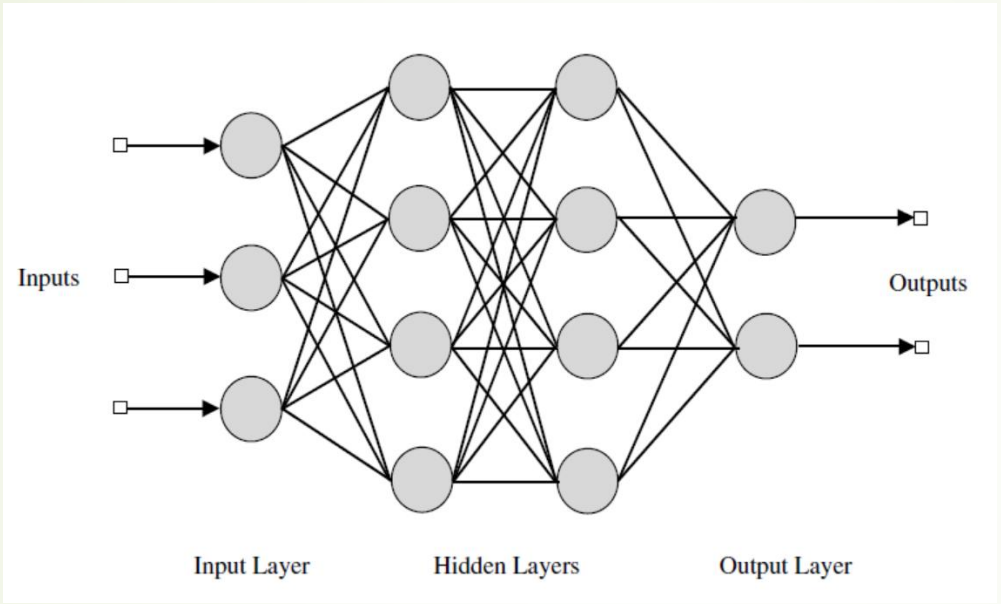
Data Mining

Multilayer Feed- Forward Neural Network

Multilayer Feed-Forward Neural Network

- A type of Artificial Neural Network (ANN)
- Composed of Input Layer, Hidden Layers, and Output Layer
- Fully connected: each node in one layer connects to all nodes in the next
- Processes data only in forward direction (no feedback loops)

Architecture



- Input Layer: Receives raw features
- Hidden Layers: Perform nonlinear transformations
- Output Layer: Produces predictions

How it works?

- Inputs go into the first layer (the input layer).
- They're transformed by weights and biases and passed through an activation function.
- This continues through one or more hidden layers.
- Finally, the output layer makes a prediction.

Input layer

- This is the entry point of the network.
- Each neuron here represents one feature from the dataset.
- The input layer does not perform any computation itself — it simply passes raw numbers forward.

Hidden layers

- These are where the real computation happens.
- Each neuron in a hidden layer does these three things:
- Weighted sum: Each input is multiplied by a weight and then all of them are summed.
- Bias: A constant bias is added to give flexibility in shifting the function.
- Activation function: A non-linear squashing function like ReLU, tanh, or sigmoid is applied. This step is what allows the network to learn complex, non-linear patterns.

Output layer

- The output layer produces the final prediction.
- Its design depends on the problem we are trying to solve:
  - Regression Problems: the output layer will contain one neuron with no activation (linear)
  - Binary Classification: output layer would be one neuron with a sigmoid activation
  - Multi-Label Classification: The output layer would have one neuron per class, with softmax activation.

Training with Backpropagation

- Forward Pass → compute predicted outputs
- Loss Function → measures prediction error (e.g., MSE, Cross-Entropy)
- Backward Pass → propagate error backwards to compute gradients
- Weight Update → use gradient descent or its variants (SGD, Adam)

Strengths

- Can model complex nonlinear relationships
- High accuracy in image, text, and speech domains
- Scalable to large problems with deep architectures
- General-purpose learner (works across domains)

Weaknesses

- Requires large labelled datasets
- Computationally expensive (needs GPUs/TPUs for deep nets)
- Training can be slow & difficult to tune
- Risk of overfitting without regularization